

# Laboratory Work No. 2

Normalization and DDL in an Airline Database

Prepared by: Emiliya Rodnova

Topic: data normalization, creation and modification of  
table structures

Almaty · 2026

# Part 1. Normalization

## Task 1. Analysis of the Source Table

The source table describes a booking, passenger, flight, airports, airline, seats, and ticket price.

```
Booking_receipt(booking_id, passenger_full_name, passenger_passport_number,  
flight_number, departure_airport_name, departure_city, arrival_airport_name,  
arrival_city, airline_name, seat_numbers, ticket_price)
```

The seat\_numbers field contains several values in one cell, for example "12A, 12B". Therefore, the source table does not satisfy the First Normal Form (1NF): each field must contain one atomic value.

After separating the seats, a composite key (booking\_id, seat\_number) is created. However, passenger, flight, airport, airline, and price information depends only on booking\_id, not on the whole key. Therefore, the table after 1NF does not satisfy 2NF.

The dependencies booking\_id → flight\_number and flight\_number → airline\_name are also given. They form a transitive dependency, so the data must be divided into separate entities to achieve 3NF.

## Task 2. Step-by-Step Normalization of Booking\_receipt

### Source Structure

| Field                                      | Purpose                    |
|--------------------------------------------|----------------------------|
| booking_id                                 | booking identifier         |
| passenger_full_name                        | passenger full name        |
| passenger_passport_number                  | passport number            |
| flight_number                              | flight number              |
| departure_airport_name /<br>departure_city | departure airport and city |
| arrival_airport_name / arrival_city        | arrival airport and city   |
| airline_name                               | airline name               |
| seat_numbers                               | list of seats              |
| ticket_price                               | ticket price               |

### 1NF

Values from seat\_numbers were split into separate rows, and the field was renamed to seat\_number.

| booking_id | passenger_full_name | flight_number | seat_number | ticket_price |
|------------|---------------------|---------------|-------------|--------------|
| 1          | Passenger A         | KC101         | 12A         | 45000        |
| 1          | Passenger A         | KC101         | 12B         | 45000        |

Primary key: (booking\_id, seat\_number).

# Normalization: 2NF and 3NF

## 2NF

In the composite key, some attributes depend only on booking\_id; these are partial dependencies. Therefore, seat information was moved to a separate Booking\_seat table.

| Booking                                | Booking_seat        |
|----------------------------------------|---------------------|
| booking_id (PK)                        | booking_id (PK, FK) |
| passenger_full_name                    | seat_number (PK)    |
| passenger_passport_number              |                     |
| flight_number                          |                     |
| departure_airport_name, departure_city |                     |
| arrival_airport_name, arrival_city     |                     |
| airline_name                           |                     |
| ticket_price                           |                     |

Relationship: Booking\_seat.booking\_id → Booking.booking\_id.

## 3NF

The Booking table retained a transitive dependency: booking\_id → flight\_number → airline\_name. To remove redundancy, the data was split into Passenger, Airline, Airport, Flight, Booking, and Booking\_seat entities.

| Table        | Main fields                                                                                        |
|--------------|----------------------------------------------------------------------------------------------------|
| Passenger    | passenger_id (PK), passenger_full_name, passenger_passport_number                                  |
| Airline      | airline_id (PK), airline_name                                                                      |
| Airport      | airport_id (PK), airport_name, city                                                                |
| Flight       | flight_id (PK), flight_number, airline_id (FK), departure_airport_id (FK), arrival_airport_id (FK) |
| Booking      | booking_id (PK), passenger_id (FK), flight_id (FK), ticket_price                                   |
| Booking_seat | booking_id (PK, FK), seat_number (PK)                                                              |

### Task 3. Functional Dependencies

In the final schema, each entity stores only its own data, and relationships are established through primary and foreign keys.

```
booking_id → passenger_id, flight_id, ticket_price
passenger_id → passenger_full_name, passenger_passport_number
flight_id → flight_number, airline_id, departure_airport_id, arrival_airport_id
airline_id → airline_name
departure_airport_id → departure_airport_name, departure_city
arrival_airport_id → arrival_airport_name, arrival_city
```

### Task 4. Checking Flights for 3NF

The primary key of the Flights table is flight\_id. All remaining fields depend on it: scheduled and actual times, airports, gates, airline, and service dates. Descriptive airport and airline data is not duplicated; only their identifiers are stored.

Conclusion: the Flights table has no transitive dependencies between non-key attributes, so it satisfies 3NF.

### Task 5. SQL Implementation of 3NF Tables

```
CREATE TABLE Passenger (
    passenger_id INT PRIMARY KEY,
    passenger_full_name VARCHAR(100) NOT NULL,
    passenger_passport_number VARCHAR(20) NOT NULL
);

CREATE TABLE Airline (
    airline_id INT PRIMARY KEY,
    airline_name VARCHAR(100) NOT NULL
);

CREATE TABLE Airport (
    airport_id INT PRIMARY KEY, airport_name VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL
);

CREATE TABLE Flight (
    flight_id INT PRIMARY KEY, flight_number VARCHAR(30) NOT NULL,
    airline_id INT NOT NULL, departure_airport_id INT NOT NULL,
    arrival_airport_id INT NOT NULL,
    FOREIGN KEY (airline_id) REFERENCES Airline(airline_id),
    FOREIGN KEY (departure_airport_id) REFERENCES Airport(airport_id),
    FOREIGN KEY (arrival_airport_id) REFERENCES Airport(airport_id)
);

CREATE TABLE Booking (
    booking_id INT PRIMARY KEY, passenger_id INT NOT NULL,
    flight_id INT NOT NULL, ticket_price DECIMAL(10,2) NOT NULL,
    FOREIGN KEY (passenger_id) REFERENCES Passenger(passenger_id),
    FOREIGN KEY (flight_id) REFERENCES Flight(flight_id)
);

CREATE TABLE Booking_seat (
    booking_id INT NOT NULL, seat_number VARCHAR(10) NOT NULL,
    PRIMARY KEY (booking_id, seat_number),
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id)
);
```

# Test Data for the 3NF Schema

To test the tables, data for airlines, airports, passengers, flights, and bookings was added.

```
INSERT INTO Airline (airline_id, airline_name) VALUES
(1, 'Air Astana'), (2, 'SCAT Airlines');
```

```
INSERT INTO Airport (airport_id, airport_name, city) VALUES
(1, 'Almaty International Airport', 'Almaty'),
(2, 'Astana International Airport', 'Astana'),
(3, 'Shymkent International Airport', 'Shymkent');
```

```
INSERT INTO Passenger (passenger_id, passenger_full_name, passenger_passport_number) VALUES
(1, 'Passenger One', 'P100001'), (2, 'Passenger Two', 'P100002'),
(3, 'Passenger Three', 'P100003'), (4, 'Passenger Four', 'P100004'),
(5, 'Passenger Five', 'P100005');
```

```
INSERT INTO Flight (flight_id, flight_number, airline_id, departure_airport_id, arrival_airport_id) VALUES
(1, 'KC101', 1, 1, 2), (2, 'KC102', 1, 2, 1),
(3, 'DV701', 2, 1, 3), (4, 'DV702', 2, 3, 1),
(5, 'KC201', 1, 1, 3);
```

```
INSERT INTO Booking (booking_id, passenger_id, flight_id, ticket_price) VALUES
(1, 1, 1, 45000.00), (2, 2, 2, 47000.00), (3, 3, 3, 35000.00),
(4, 4, 4, 37000.00), (5, 5, 5, 42000.00);
```

```
INSERT INTO Booking_seat (booking_id, seat_number) VALUES
(1, '12A'), (1, '12B'), (2, '15C'), (3, '7A'), (4, '10B'), (5, '18F');
```

Result: the normalized structure is implemented in SQL; relationship integrity is maintained by PK and FK constraints.

## Part 2. DDL

### Task 1. Creating Tables

According to the task, tables were created to store information about airlines, airports, passengers, flights, bookings, baggage, and security checks.

```
CREATE TABLE Airline_info (airline_id INT, airline_code VARCHAR(30), airline_name VARCHAR(50), airline_country VARCHAR(50), created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Airport (airport_id INT, airport_name VARCHAR(50), country VARCHAR(50), state VARCHAR(50), city VARCHAR(50), created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Baggage_check (baggage_check_id INT, check_result VARCHAR(50), created_at TIMESTAMP, updated_at TIMESTAMP, booking_id INT);
CREATE TABLE Baggage (baggage_id INT, weight_in_kg DECIMAL(4,2), created_at TIMESTAMP, updated_at TIMESTAMP, booking_id INT);
CREATE TABLE Boarding_pass (boarding_pass_id INT, booking_id INT, seat VARCHAR(50), boarding_time TIMESTAMP, created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Booking_flight (booking_flight_id INT, booking_id INT, flight_id INT, created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Booking (booking_id INT, flight_id INT, passenger_id INT, booking_platform VARCHAR(50), created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Flights (flight_id INT, sch_departure_time TIMESTAMP, sch_arrival_time TIMESTAMP, departing_airport_id INT, arriving_airport_id INT);
CREATE TABLE Passengers (passenger_id INT, first_name VARCHAR(50), last_name VARCHAR(50), date_of_birth DATE, gender VARCHAR(50), created_at TIMESTAMP, updated_at TIMESTAMP);
CREATE TABLE Security_check (security_check_id INT, check_result VARCHAR(20), created_at TIMESTAMP, updated_at TIMESTAMP, passenger_id INT);
```

## Task 2. Primary Keys

A primary key was added to each table to uniquely identify every record.

```
ALTER TABLE Airline_info ADD PRIMARY KEY (airline_id);
ALTER TABLE Airport ADD PRIMARY KEY (airport_id);
ALTER TABLE Baggage_check ADD PRIMARY KEY (baggage_check_id);
ALTER TABLE Baggage ADD PRIMARY KEY (baggage_id);
ALTER TABLE Boarding_pass ADD PRIMARY KEY (boarding_pass_id);
ALTER TABLE Booking_flight ADD PRIMARY KEY (booking_flight_id);
ALTER TABLE Booking ADD PRIMARY KEY (booking_id);
ALTER TABLE Flights ADD PRIMARY KEY (flight_id);
ALTER TABLE Passengers ADD PRIMARY KEY (passenger_id);
ALTER TABLE Security_check ADD PRIMARY KEY (security_check_id);
```

## Task 4. NOT NULL Constraints

The NOT NULL constraint was set for all required non-key attributes. Primary key fields also cannot contain NULL values.

```
ALTER TABLE Airline_info
  ALTER COLUMN airline_code SET NOT NULL,
  ALTER COLUMN airline_name SET NOT NULL,
  ALTER COLUMN airline_country SET NOT NULL,
  ALTER COLUMN created_at SET NOT NULL,
  ALTER COLUMN updated_at SET NOT NULL,
  ALTER COLUMN info SET NOT NULL;

ALTER TABLE Airport
  ALTER COLUMN airport_name SET NOT NULL, ALTER COLUMN country SET NOT NULL,
  ALTER COLUMN state SET NOT NULL, ALTER COLUMN city SET NOT NULL,
  ALTER COLUMN created_at SET NOT NULL, ALTER COLUMN updated_at SET NOT NULL;

-- The same required-field rule is set for Baggage_check, Baggage,
-- Boarding_pass, Booking_flight, Booking, Flights, Passengers,
-- and Security_check for all listed non-key fields.
```

# Modifying Table Structures

## Task 5. Renaming a Table

The Airline\_info table was renamed to Airline.

```
ALTER TABLE Airline_info RENAME TO Airline;
```

## Task 6. Renaming a Column

In the Booking table, the price column was renamed to ticket\_price.

```
ALTER TABLE Booking RENAME COLUMN price TO ticket_price;
```

## Task 7. Changing a Data Type

The data type of the departing\_gate field in Flights was changed from VARCHAR(50) to TEXT.

```
ALTER TABLE Flights ALTER COLUMN departing_gate TYPE TEXT;
```

## Task 8. Deleting a Column

The info column was removed from Airline because it is no longer used.

```
ALTER TABLE Airline DROP COLUMN info;
```

## Task 9. Creating Relationships Between Tables

Foreign keys establish relationships between passengers, bookings, flights, airports, and airlines.

```
ALTER TABLE Security_check ADD CONSTRAINT fk_security_passenger FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id);
ALTER TABLE Booking ADD CONSTRAINT fk_booking_passenger FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id);
ALTER TABLE Baggage_check ADD CONSTRAINT fk_baggage_check_passenger FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id);
ALTER TABLE Baggage_check ADD CONSTRAINT fk_baggage_check_booking FOREIGN KEY (booking_id) REFERENCES Booking(booking_id);
ALTER TABLE Baggage ADD CONSTRAINT fk_baggage_booking FOREIGN KEY (booking_id) REFERENCES Booking(booking_id);
ALTER TABLE Boarding_pass ADD CONSTRAINT fk_boarding_pass_booking FOREIGN KEY (booking_id) REFERENCES Booking(booking_id);
ALTER TABLE Booking_flight ADD CONSTRAINT fk_booking_flight_booking FOREIGN KEY (booking_id) REFERENCES Booking(booking_id);
ALTER TABLE Booking_flight ADD CONSTRAINT fk_booking_flight_flight FOREIGN KEY (flight_id) REFERENCES Flights(flight_id);
ALTER TABLE Flights ADD CONSTRAINT fk_flights_departure_airport FOREIGN KEY (departing_airport_id) REFERENCES Airport(airport_id);
ALTER TABLE Flights ADD CONSTRAINT fk_flights_arrival_airport FOREIGN KEY (arriving_airport_id) REFERENCES Airport(airport_id);
ALTER TABLE Flights ADD CONSTRAINT fk_flights_airline FOREIGN KEY (airline_id) REFERENCES Airline(airline_id);
```

Overall conclusion: the source table was normalized to 3NF, and the database structure was created and modified using DDL. All necessary primary keys, foreign keys, and integrity constraints were defined.

**Prepared by: Emiliya Rodnova**